# BriskSnapshot: A Live Demo of Join-Backed Semantic Windows for Streaming AI Pipelines

Anonymous Author(s)

## Abstract

Fresh semantic state is becoming a first-class systems requirement for large-model applications. Operational copilots, streaming retrieval pipelines, and long-running agents all need intermediate vector state that can be updated continuously and consumed immediately by downstream reasoning steps. However, today this state is often reconstructed outside the runtime that observes the stream, making the resulting application behavior difficult to explain, reuse, and demonstrate. We present BriskSnapshot, a live demonstration of a unified streaming AI stack that keeps semantic-window state inside the runtime while exposing compact, application-readable contracts to downstream services. In the demo, BriskSnapshot replays a multi-source vulnerability-intelligence stream, shows how orchestration operators hand events to a persistent semantic-window runtime, and visualizes how maintained runtime state becomes vulnerability snapshots, routing decisions, and security escalation signals. A deterministic prepared run groups 24 events from five source classes into six semantic clusters and 18 snapshot insights, giving attendees a concrete way to inspect the relationship between runtime state, bounded summaries, and LLM-facing application behavior.

## CCS Concepts

• **Computer systems organization** → **Parallel architectures**; • **Information systems** → **Stream management**; • **Computing methodologies** → *Artificial intelligence.*

## Keywords

stream processing, vector databases, AI middleware, multicore systems, demonstration

## 1 Introduction

Large-model systems are increasingly assembled as continuous dataflows instead of isolated request-response programs. New observations arrive between prompts, retrieved evidence changes while a task is in progress, and operational events must be clustered before an agent or copilot can react. In each case, the application needs fresh intermediate state rather than stale batch snapshots. Existing stream processors provide scalable execution [1], and vector indexes provide efficient nearest-neighbor search [2, 3], but an end-to-end AI stack still needs a practical way to maintain join-backed semantic state under sliding windows and continuous insertions.

This demo addresses that gap with BriskSnapshot, a unified software stack that combines application-level dataflow orchestration with a multicore vector-stream runtime. The specific problem is interface reliability: how can one system accept live vector events, maintain streaming semantic state under a bounded window, evaluate each new event against that state with a real window join, and return stable, application-readable summaries without forcing the upper pipeline to manage low-level runtime details?

Another difficulty comes from full-stack coordination. Runtime state ownership, Python bindings, operator orchestration, and application-visible output must all agree on what the live semantic window means and when its join results should be consumed. Without such an agreement, upper-layer operators either reconstruct state on their own or depend on brittle runtime-specific callbacks, which makes live execution harder to reason about and harder to demonstrate.

Contributions. We present BriskSnapshot, a concrete system that exposes live semantic state as a first-class join-backed window service for LLM inference pipelines. First, we integrate a runtime-owned window service into the same boundary used by the orchestration flow. Second, we derive bounded cluster, neighbor, and novelty summaries from those runtime matches so that higher-level operators consume a stable application contract rather than raw engine internals. Third, we package the prepared demo run as a reproducible artifact, including a generated result figure derived from the real JSON output of the live execution. The result is a compact but fully operational demonstration of how multicore streaming machinery can be surfaced as application-visible AI middleware.

The rest of the paper proceeds from problem setting to system design and then to the live demonstration path. We first introduce the system-level motivation behind join-backed semantic windows, then summarize the main architectural path used by BriskSnapshot, and finally present the concrete scenario and takeaways shown in the demonstration.

## 2 Background and Problem Overview

The core problem behind BriskSnapshot is not simply fast vector search. The harder systems problem is how to maintain fresh semantic state under continuous updates and expose that state to higher-level operators in a form that is stable enough for application logic. In a live AI pipeline, the state owner, the export interface, and the orchestration path all matter. If any of these are poorly chosen, the system becomes difficult to reason about, difficult to demonstrate, and difficult to reuse.

Our implementation makes this challenge visible because the operator layer, service layer, Python binding layer, and multicore runtime inside BriskSnapshot must all agree on what a "semantic window" means. This requirement leads to four practical design constraints.

First, the runtime must own the authoritative window state. If upper-layer operators reconstruct the window independently, they can drift from the true runtime view under future scheduling, reordering, or backpressure changes. Second, the service boundary must survive repeated use. Repeated execution requires reset behavior, cleanup behavior, and long-lived runtime ownership. Third, the exported state must have the right granularity. Exporting only final alerts hides too much of the semantic state, while exporting full internal structures leaks implementation details. Fourth, the

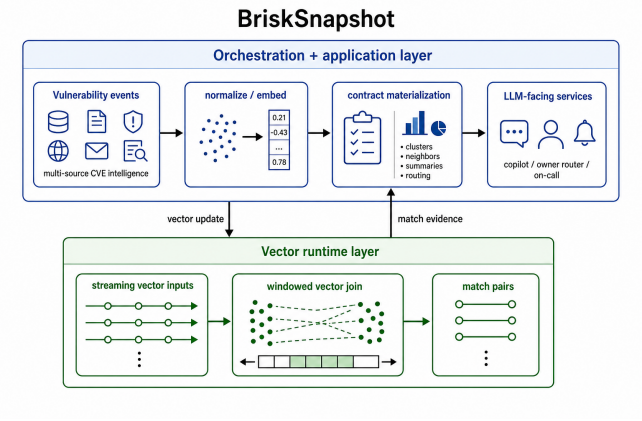


**Figure 1: System overview of BriskSnapshot. The orchestration and application layer sends vector updates into a persistent runtime, receives match evidence back, and then materializes bounded contracts for LLM-facing services.**

system must remain observable. Runtime behavior, exported state, and final application output should remain easy to relate within one execution.

These requirements shaped the current BriskSnapshot design. We therefore keep runtime ownership inside a persistent join runtime, use real window join to discover semantic matches, and expose bounded summaries rather than raw engine structures. This focus explains why the paper centers join-backed semantic windows rather than a broader collection of streaming operators.

## 3 System Overview

Figure 1 summarizes the architecture we implemented. The front half of BriskSnapshot is an application pipeline built from a batch source, a normalization step, a window-join step, an insight derivation step, and a sink. The critical operator is the window-join step: it invokes a runtime-local endpoint registered through the system's service boundary. That service owns a long-lived semantic-window runtime, backed by streaming inputs and one continuously running execution graph. This control path lets BriskSnapshot expose live semantic state to higher-level operators while keeping runtime state ownership in one place.

### 3.1 Execution Path

The execution path is intentionally narrow. A prepared batch source replays the vulnerability-intelligence stream, the normalization step resolves each record into one uniform event type, and the window-join step forwards that event to the long-lived runtime through the local service registry. This design keeps orchestration logic simple: the upper pipeline never allocates short-lived runtime objects, and it never reconstructs window state outside the runtime. Instead, each event traverses one control path from source, to runtime update, to bounded summary, to insight derivation.

This path also matches what the audience sees in the live demo. The operator graph is small enough to explain quickly, but every stage performs real work. The source and map stages run in the orchestration layer, the stateful update and window join happen inside the runtime layer, and the final sink emits the same JSON artifact that is later summarized in Figure 3. The chapter therefore describes one cohesive system path rather than a bridge between two independent products.

### 3.2 Join Runtime and State Materialization

Inside the join runtime, BriskSnapshot uses native streaming operators rather than a Python-side similarity scan. At service startup, the adapter creates one persistent runtime with dual streaming inputs, a windowed similarity join, and a sink that records emitted join pairs. For each arriving event, the orchestration layer appends a record to the runtime inputs and reads the newly emitted matches from that same running graph. Those matches return as vector evidence; the orchestration layer then folds them into bounded neighbor, cluster, and novelty summaries so that upper-layer operators receive a stable application contract while similarity discovery remains inside the streaming engine.

The design goal is to make the runtime the single owner of semantic matching. After each insertion, the adapter materializes the current active window, invokes the join operator against that bounded set, and derives a window summary containing cluster count, source breakdown, latest event identifier, nearest neighbors, and per-cluster statistics. In the current prototype, cluster construction follows connected components over the returned join matches, and the window summary is sorted by cluster size and average severity before it is exposed to the application. This gives the upper pipeline a compact state summary without exposing lower-level runtime structures.

The trade-off is deliberate. We do not export mutable internal indexes or rely on Python-level callbacks to represent authoritative similarity state. Instead, we keep match discovery inside the runtime and export only a bounded summary derived from the join results. This choice keeps the demo deterministic and debuggable while still leaving a real multicore runtime on the critical path.

This design matters because it removes the system's correctness dependency on a Python-side reimplementation of similarity discovery. The live path still uses real ingestion, a bounded active window, and deterministic summary generation, but the match computation itself remains inside the runtime. That trade-off keeps a real multicore runtime in the loop while preserving predictable and repeatable behavior.

### 3.3 Insight Materialization

The last part of the chapter is where BriskSnapshot turns runtime state into application-visible evidence. For every event, the adapter computes nearest neighbors from the join result, derives a novelty score from the top similarity, and identifies the event's active cluster in the current summary. The downstream insight stage then applies explicit rules to this state: a cluster becomes a correlated incident when it contains at least three events, has average severity at least 0.70, and spans at least two sources; an event becomes an emerging pattern when its novelty score is at least 0.18 and its severity is at least 0.95. These rules are simple, but they are sufficient to show how join-backed semantic state can drive concrete next-step decisions.

**Table 1: Responsibilities across the BriskSnapshot stack.**

| Layer | Responsibility |
|---|---|
| BriskSnapshot operators | Event normalization, service invocation, insight derivation, final sink output |
| Window-join access layer | Runtime ownership, reset and cleanup, stable application-facing summary access |
| Python binding | Record ingestion, join invocation, and result handoff into the application process |
| Join-backed window engine | Active-window materialization, bounded join execution, streaming state ownership |

The architecture therefore illustrates a useful software boundary for AI infrastructure. Rather than forcing high-level operators to embed vector-state logic directly, BriskSnapshot keeps high-frequency state updates and window joins inside its runtime engine and exposes just enough summarized state to downstream operators through a stable system contract. The benefit is not only modularity. It also makes the effect of runtime parameters legible at the application layer: changing similarity threshold, active-window size, or event order does not just perturb an internal data structure, it changes which incidents are correlated, which anomalies remain novel, and what evidence the next inference stage receives.

Table 1 summarizes this division of responsibility. It makes clear that BriskSnapshot is one software stack with distinct internal roles rather than two separately described subsystems.

## 4 Demonstration Scenario

The demo uses a deterministic public vulnerability-intelligence stream implemented in the current prototype. Twenty-four vector events arrive from NVD, vendor advisories, CISA KEV entries, CISA alerts, and research notes. They describe six evolving vulnerability families, including PAN-OS, XZ Utils, OpenSSH regreSSHion, PHP-CGI, Fortinet SSL-VPN, and TeamCity. Each event has an embedding, severity score, tags, and metadata. BriskSnapshot ingests these updates continuously, maintains a live semantic window, and emits join-backed summaries that can be consumed immediately by downstream LLM inference services.

The downstream operators then turn those bounded summaries into application-facing insights. In the prepared run, cross-source CVE families provide fresh prompt context and routing evidence, while high-severity singleton updates become emerging anomalies for a security-facing agent. Figure 2 is therefore not an internal pipeline sketch; it shows the end-to-end service role of BriskSnapshot inside an LLM inference stack, where runtime-maintained semantic state is surfaced as bounded evidence for the next inference step.

Figure 3 summarizes the concrete outcome of one deterministic prepared run. To avoid presenting this as a hand-written schematic, the figure is generated from the JSON artifact emitted by the real demo execution and included as a reproducible prepared-run record.

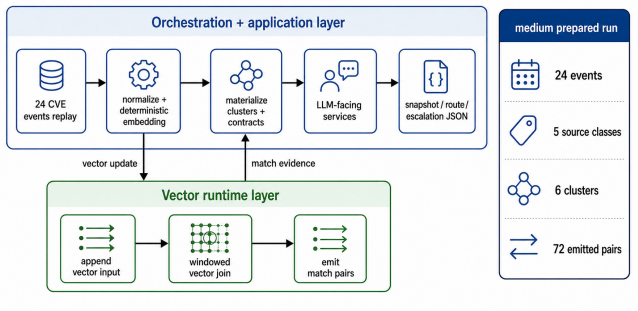


**Figure 2: Live demonstration flow. The prepared replay sends vector updates into the runtime, receives match evidence back, and materializes snapshot, routing, and escalation outputs in the orchestration layer.**

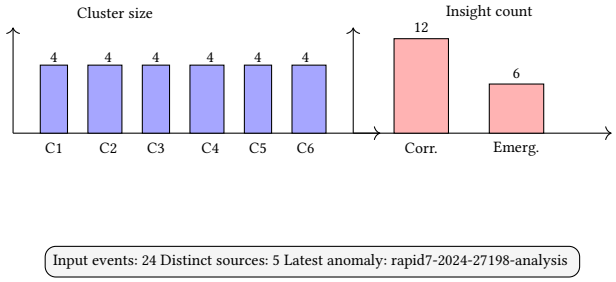


**Figure 3: Result summary of one deterministic prepared BriskSnapshot run, generated from the JSON artifact emitted by the live demo.**

The current implementation produces six clusters and 18 snapshot insights from the 24-event stream. Each cluster corresponds to a cross-source vulnerability family, and the companion routing and escalation paths export route and on-call contracts over the same maintained state. This is not a throughput benchmark; it is a compact run summary that connects runtime state, join-backed summaries, and final application output in one execution.

### 4.1 Demonstration Flow

The live demonstration highlights a practical systems lesson: a usable AI pipeline needs a robust runtime boundary, not just a powerful operator set. The path emphasized here keeps the true runtime on the ingestion path while returning a bounded summary through a narrow join-backed interface. This makes the runtime-managed semantic window visible without making execution fragile.

The live demonstration follows one end-to-end path instead of several loosely connected micro-scenarios. It begins with the BriskSnapshot operator graph and identifies the exact point where the join runtime is invoked through the query path. This makes the software boundary explicit: one part of BriskSnapshot handles orchestration, while another part owns runtime state.

It then runs the incident stream replay and exposes the real runtime output. The execution log, bounded summary payload, and final JSON artifact remain visible in the same path, making clear

how a multicore stream runtime behaves when embedded inside a higher-level application stack rather than evaluated in isolation.

- **Persistent join runtime.** The demo highlights how a long-lived runtime object can expose window join through one stable control path instead of scattering ad hoc low-level calls throughout the application.
- **Stable bounded summary.** We show the runtime-level join path and explain why a bounded summary API is the right live-demo abstraction for application-facing semantic state.
- **Application semantics.** We connect the low-level join result to the final user-visible result: correlated incident detection and anomaly surfacing from multi-source vector events.
- **Parameter sensitivity.** By varying active-window size, similarity threshold, and event ordering in prepared runs, we can show how cluster composition and novelty scores change at the application layer.

This interaction model links parallel runtime structure to application-visible behavior without requiring a large deployment or a specialized dataset.

## 4.2 Prepared Demo Variants

Although the paper centers one stable end-to-end path, the live session can still show several meaningful variants without changing the software architecture. The first variant is the medium replay used throughout this paper. It uses the prepared 24-event stream, a moderate window size, and the current similarity threshold. This run surfaces six correlated vulnerability clusters and 18 snapshot insights. It is a useful entry point because it exposes every layer of BriskSnapshot in a compact execution.

Table 2 summarizes the three most informative deviations from this baseline. Raising the similarity threshold fragments the final state and suppresses several correlated outputs. Reducing active-window size removes older vulnerability families from the exported state and shrinks active source coverage. Reversing the same event stream preserves the topic families but changes the latest event and the order in which application-visible insights are emitted. Together, these runs expose retention, semantic strictness, and order sensitivity as concrete system-level trade-offs.

These are small experiments, but they expose exactly the state-management trade-offs that matter for parallel runtimes embedded in AI inference systems.

**Table 2: Observed results of prepared variants executed on the live demo path.**

| Variant | Runtime setting | Observed result |
|---|---|---|
| Medium replay | window=28, threshold=0.985 | Final summary: 6 clusters of size 4; snapshot output: 18 correlated insights |
| Aggressive threshold | window=28, higher threshold | Final summary fragments into more smaller clusters; correlated snapshot output drops |
| Shorter active window | window=12, threshold=0.985 | Older vulnerability families leave the exported state; active source coverage shrinks |
| Reordered stream | window=28, threshold=0.985, reversed order | Topic families remain visible, but latest event and insight emission order change |

# 5 Conclusion

This paper presents an ICPP demo built around BriskSnapshot, a concrete system that unifies streaming orchestration, vector-state maintenance, and join-backed semantic summaries. The central idea is to treat live vector-state management as a first-class system capability rather than as an isolated kernel or an external component. The current implementation already demonstrates this path on a persistent semantic-window runtime using a deterministic multi-source vulnerability-intelligence stream. In the prepared run used throughout the demo, BriskSnapshot turns 24 events from five source classes into six semantic clusters and 18 snapshot insights while keeping the summary path stable and application-readable.

More broadly, the demo matters because it turns interface design for parallel runtimes into an application-visible systems problem. BriskSnapshot shows that multicore stream processing is not only about accelerating kernels; it is also about choosing a state owner, choosing a summary granularity, and providing an orchestration path that keeps the full stack understandable under live execution. The resulting system offers a compact but meaningful example of how parallel stream-processing mechanisms can be surfaced as practical AI middleware. An immediate next step is to expand the prepared scenario set and evaluate how active-window size, similarity thresholds, and arrival-order changes affect the quality and timeliness of exported evidence for downstream inference services.

# References

[1] Tyler Akidau, Alex Balikov, Kaya Bekiroğlu, Slava Chernyak, Josh Haberman, Reuven Lax, Sam McVeety, Daniel Mills, Paul Nordstrom, Sam Whittle, Robert Wilfong, and Lukasz Zając. 2015. The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing. *Proceedings of the VLDB Endowment* 8, 12 (2015), 1792–1803.

[2] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33, 1 (2011), 117–128.

[3] Yu A. Malkov and D. A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42, 4 (2020), 824–836.